Blockchain Lab 02: Keys and Cryptography

Student implementation and evidence report

Prepared from my Python experiments and saved PowerShell output. This report documents Parts A-D, the required security demonstrations, and the professor's verification tests.

Repository: github.com/korneliasara/blockchain-lab02-keys-and-crypto

Component	Evidence
Part A	Group-law verification and public-key operations
Part B	ECDSA signatures, RFC 6979, and nonce-reuse experiment
Part C	BIP-39/32/44 accounts and parent-key recovery
Part D	Post-quantum size and throughput comparison
Verification	64/64 reference tests; 81/81 personal tests

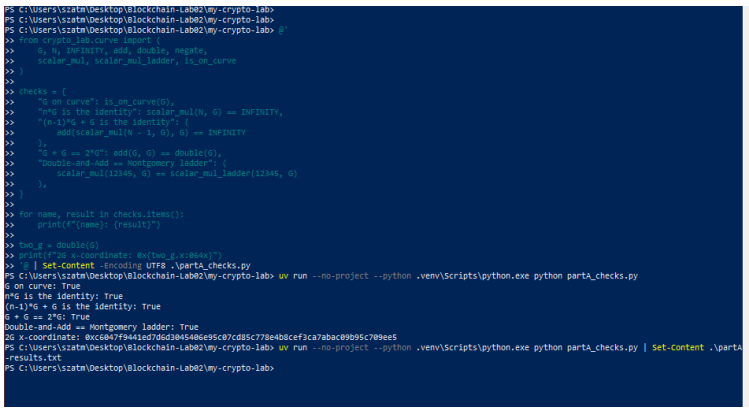
All cryptographic secrets used in these demonstrations are publicly known example values, not real wallet credentials. My Python implementation is for educational purposes, not real asset custody.

Part A - Elliptic-curve group-law verification

I implemented the secp256k1 curve operations myself: modular inversion, point addition, doubling, negation, scalar multiplication by Double-and-Add, and a ladder algorithm. I checked the results against the published coordinates of 2G and the group identity.

Check	Observed result
G satisfies the curve equation	True
nG is the point at infinity	True
(n-1)G + G is the identity	True
G + G = 2G	True
Double-and-Add equals ladder (sample scalar)	True

The special point at infinity behaves like zero when adding points. Adding a point and its negative must return infinity. The generator also has order n, so adding it n times returns infinity. These tests helped me confirm that the arithmetic follows the curve rules. Both multiplication methods produced the same answer in my recorded check. The professor's unchanged tests also passed; these include edge cases and scalar multiplication equivalence.



Part A: group-law verification in PowerShell (illustrative terminal evidence).

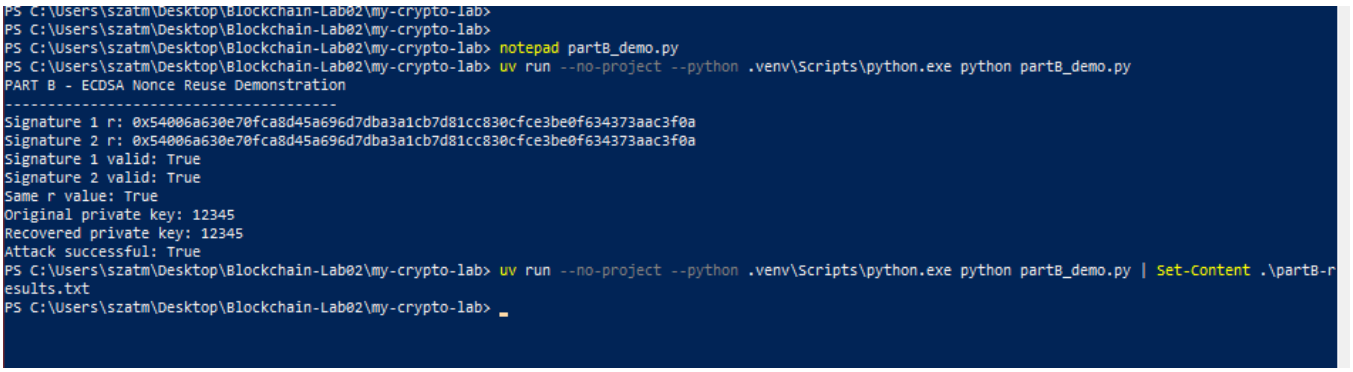
Part B - ECDSA, deterministic nonce, and the nonce-reuse attack

I implemented signature creation and verification, RFC 6979 deterministic nonce generation, low-s normalization, and public-key recovery. RFC 6979 derives a nonce from the private key and message hash. This avoids depending on fresh random-number generation for every signature, and it helps prevent accidental nonce reuse caused by a weak random generator.

To see why nonce reuse is dangerous, I signed two different messages with the same demonstration nonce. Both signatures verified, but they had the same r value. Using the two message hashes and signature values, I recovered the original private key.

Demonstration field	Observed value
First signature valid	True
Second signature valid	True
Same r value	True
Original example private key	12345
Recovered private key	12345
Attack successful	True

With the same nonce k, subtracting the two ECDSA signature equations makes it possible to calculate k and then the private key. I also updated the recovery function to handle low-s normalization, which can replace s with n-s. The professor's low-s nonce-reuse test passed.

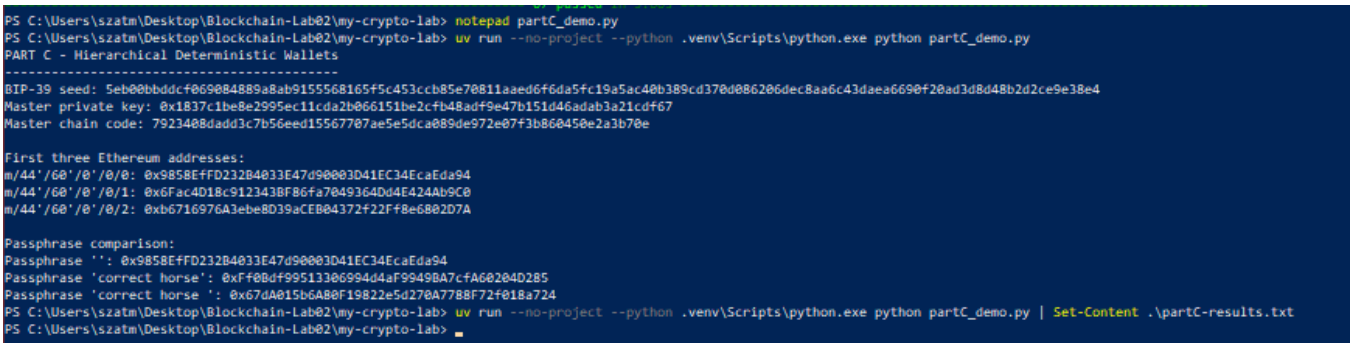


Part B: nonce-reuse demonstration screenshot (example key only).

Part C - Wallet derivation and parent-key exposure

I derived a BIP-39 seed from the published "abandon ... about" test phrase, generated a BIP-32 master key and chain code, then followed the Ethereum BIP-44 path m/44'/60'/0'/0/i. This gave three separate account addresses for indices 0, 1, and 2. I used Keccak-256 and EIP-55 capitalization to produce the addresses.

The published BIP-39 seed and Ethereum address vectors were checked by the professor's reference tests. I also tested an empty passphrase, "correct horse", and "correct horse " with a final space. Each passphrase led to a different wallet, illustrating why an exact recovery phrase *and* passphrase are required.



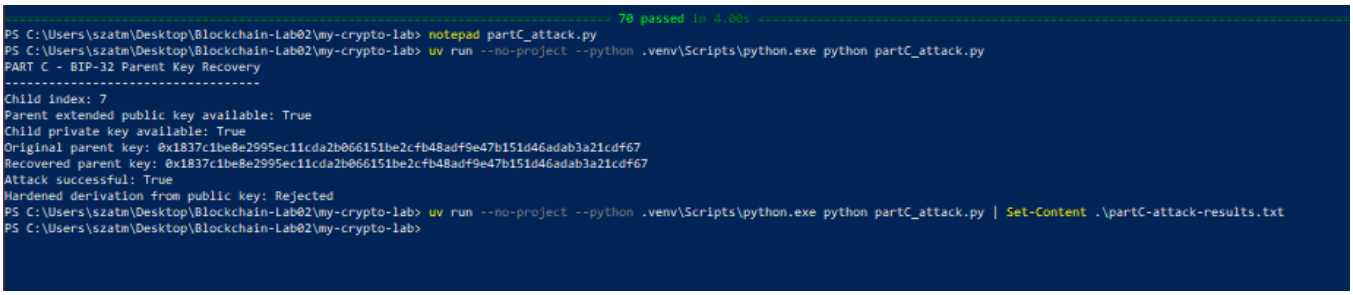
Part C: first three addresses and BIP-39 passphrase comparison.

Part C - Parent-key recovery continued

For the non-hardened child at index 7, I deliberately exposed the parent extended public information and the child private key. The recovery calculation returned exactly the original parent private key. This works because non-hardened derivation adds a tweak that is computable from the extended public key.

Hardened derivation instead uses the parent private key in the derivation data. Someone with only the parent extended public key cannot produce the same hardened child or apply that particular parent-key recovery attack. My attempt to derive a hardened child from the public-only parent was rejected.

Parent-key experiment	Observed result
Non-hardened child index	7
Recovered parent equals original	True
Hardened derivation from public parent	Rejected



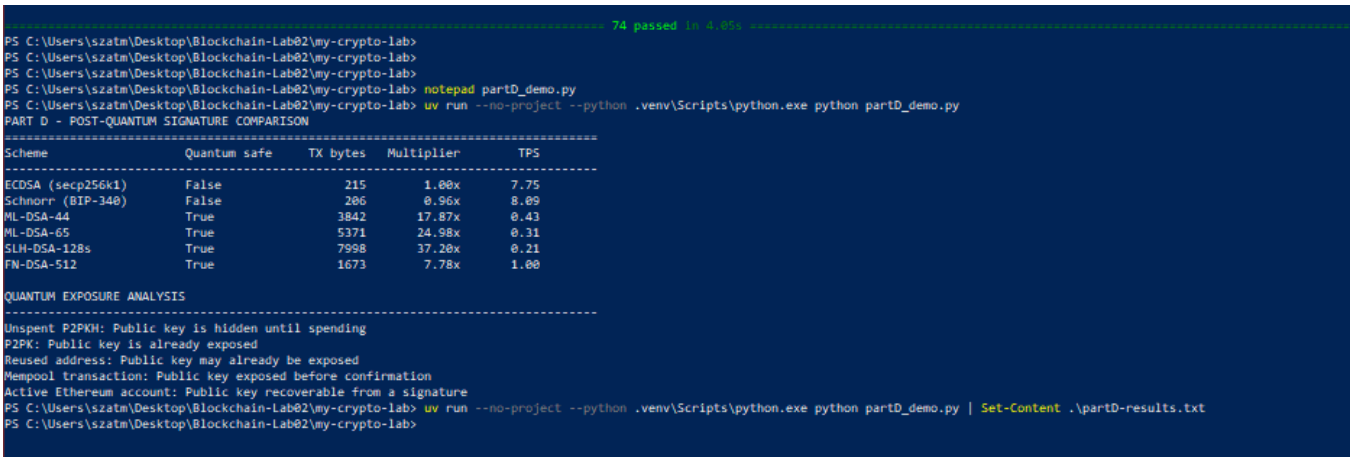
Part C: parent-private-key recovery demonstration.

Part D - Post-quantum signature comparison

I compared the approximate public-key and signature sizes of current elliptic-curve signatures with post-quantum alternatives. To keep the comparison understandable, the program uses a simplified transaction model with 110 bytes of overhead and assumes a fixed block capacity. The table below is not a measurement of the Bitcoin network.

Scheme	TX bytes	vs. ECDSA	Est. TPS
ECDSA	215	1.00x	7.75
Schnorr	206	0.96x	8.09
ML-DSA-44	3,842	17.87x	0.43
ML-DSA-65	5,371	24.98x	0.31
SLH-DSA-128s	7,998	37.20x	0.21
FN-DSA-512	1,673	7.78x	1.00

The clear trade-off is that larger signatures allow fewer transactions in the same block budget. For example, ML-DSA-44 makes the simplified transaction about 17.9 times larger than ECDSA. Quantum migration is not only about choosing a secure algorithm; block space, wallets, verification, compatibility, and users also matter.



Part D: generated signature and throughput comparison.

Part D - A possible fifteen-year migration

Years 1-3: Study quantum exposure, track standards, test candidate schemes, and inventory systems using elliptic-curve signatures. Prioritize places where the public keys are already visible.

Years 4-7: Introduce optional post-quantum or hybrid signature paths in wallet and network software. Perform audits and test how larger signatures affect block space, transaction fees, hardware wallets, and verification time.

Years 8-11: Encourage migration of old balances to quantum-resistant outputs and provide clear instructions, including for offline or rarely used accounts. Keep a long transition period so people do not lose access to their money.

Years 12-15: Review remaining vulnerable outputs and decide how legacy rules should be handled through the community's consensus process. Continue checking new cryptographic research and update the design if weaknesses appear. This is an illustrative timetable, not a prediction of when useful quantum computers will exist.

Which funds are exposed? Bitcoin P2PK outputs publish the public key directly. A P2PKH public key is normally hidden until a spend reveals it, while address reuse can expose a remaining balance. Transactions waiting in the mempool expose public keys before confirmation. Ethereum accounts that have signed transactions also reveal enough information to recover their public keys. This creates a "harvest now, attack later" concern: public cryptographic material recorded today remains available to an attacker in the future.

Implementation risks (under 400 words)

Even if an ECDSA implementation returns signatures that verify correctly, it can still leak a private key in several ways.

1. Repeated or predictable nonce. If two different messages reuse the same nonce, their signatures can reveal the private key. A weak random-number generator can make nonces guessable too. The mitigation is using a correctly implemented deterministic nonce method, such as RFC 6979, and never intentionally reusing a nonce. My own `sign()` function still accepts an explicit nonce for demonstrations, so someone could misuse it, as I showed in Part B.

2. Timing and power side channels. Different private-key bits can cause different operations or execution times. My `scalar_mul()` Double-and-Add function branches when a bit is set. I also wrote a ladder variant, but Python big-integer arithmetic and branching are not constant-time. A real attacker who repeatedly measures these differences could learn secret information. The mitigation is to use audited, constant-time cryptographic code and reduce observable side channels.

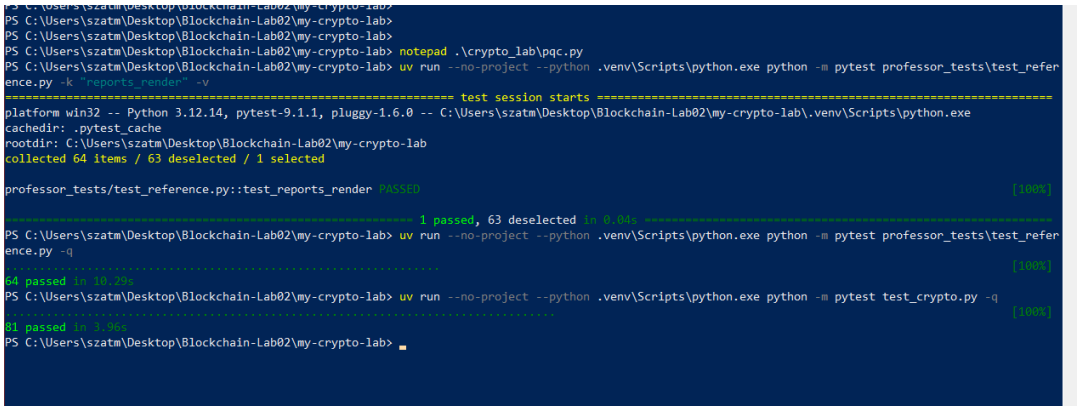
3. Biased or partially leaked nonces. Even if the nonce never repeats exactly, leaking a few nonce bits across many signatures can allow mathematical attacks against the signing key. Mitigations include sound nonce generation, keeping secrets out of debug logs, and using hardened, audited signing implementations.

These weaknesses show why correct mathematical results and passing tests are not enough to make my educational implementation safe for real funds.

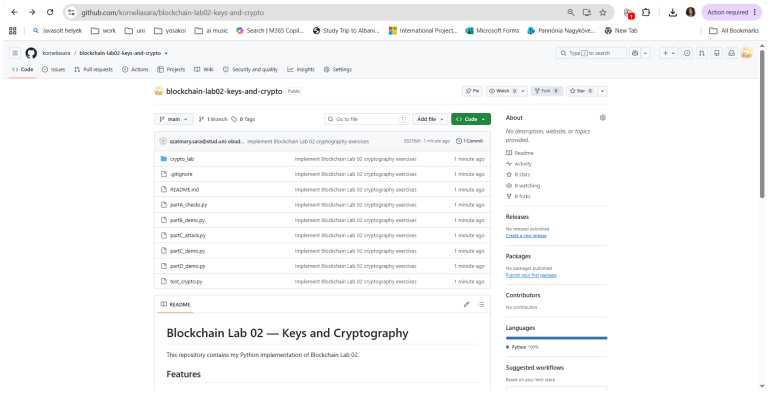
Verification and submission

I ran the professor's unchanged 64-test reference suite and my separate 81-test suite. Both completed without failed tests. The repository contains my own implementation and test code; the professor's tests were kept outside the upload.

Suite	Observed outcome
Professor reference suite	64 passed
My own suite	81 passed



PowerShell reference-suite and local-test summary.



GitHub repository with my source code and README.

Submission repository: https://github.com/korneliasara/blockchain-lab02-keys-and-crypto